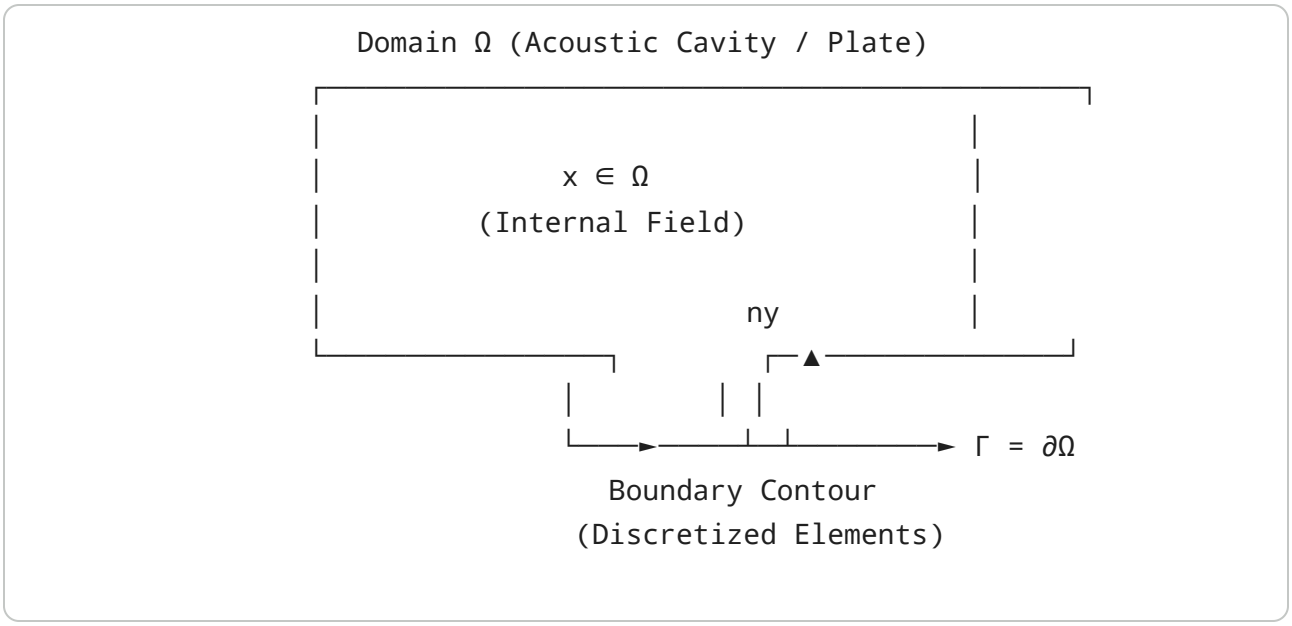
1. Mathematical Formulation: 2D Helmholtz Boundary Integral Equations

When acoustic plates or vocal tract cross-sections possess non-canonical geometries (e.g., pyriform sinuses, labial apertures, asymmetric vocal folds), separation of variables in Cartesian or polar coordinates fails. The **Boundary Element Method (BEM)** reduces the governing partial differential equation in the domain $\Omega \subset \mathbb{R}^2$ to an integral equation over the closed 1D bounding contour $\Gamma = \partial\Omega$.



The 2D Helmholtz Boundary Integral Equation

For steady-state harmonic acoustic pressure or transverse plate velocity potential $\psi(\mathbf{x})e^{-i\omega t}$ with wavenumber $k = \omega/c$, the field satisfies:

$$\left(\nabla^2 + k^2\right)\psi(\mathbf{x}) = 0, \quad \mathbf{x} \in \Omega$$

Using Green's second identity, the interior field $\psi(\mathbf{x})$ at any point $\mathbf{x} \in \Omega$ is determined by the boundary values of the potential $\psi(\mathbf{y})$ and its outward normal derivative $q(\mathbf{y}) = \frac{\partial \psi}{\partial n_y}(\mathbf{y})$:

$$c(\mathbf{x})\psi(\mathbf{x}) = \int_\Gamma \left[G(\mathbf{x}, \mathbf{y}; k) q(\mathbf{y}) - \frac{\partial G}{\partial n_y}(\mathbf{x}, \mathbf{y}; k)\psi(\mathbf{y}) \right] d\Gamma(\mathbf{y})$$

where the free-term coefficient $c(\mathbf{x})$ is:

$$c(\mathbf{x}) = \begin{cases} 1, & \mathbf{x} \in \Omega \setminus \Gamma \\ \frac{1}{2}, & \mathbf{x} \in \Gamma \quad \text{(for smooth boundaries)} \\ 0, & \mathbf{x} \notin \bar{\Omega} \end{cases}$$

2D Free-Space Fundamental Solution

The free-space Green's function $G(\mathbf{x}, \mathbf{y}; k)$ in 2D is given by the zero-order Hankel function of the first kind:

$$G(\mathbf{x}, \mathbf{y}; k) = \frac{i}{4} H_0^{(1)}(kr), \quad r = \|\mathbf{x} - \mathbf{y}\|$$

Its directional derivative along the outward unit normal $\mathbf{n}_y$ is:

$$\frac{\partial G}{\partial n_y}(\mathbf{x}, \mathbf{y}; k) = \nabla_y G \cdot \mathbf{n}_y = -\frac{ik}{4} H_1^{(1)}(kr) \frac{(\mathbf{y} - \mathbf{x}) \cdot \mathbf{n}_y}{r}$$

Boundary Discretization & Collocation

Dividing the boundary Γ into N constant boundary elements Γ_j ($j = 1, \ldots, N$) with element midpoints $\mathbf{x}_i \in \Gamma_i$ yields the linear algebraic system:

$$\mathbf{H}\boldsymbol{\psi} = \mathbf{G}\mathbf{q}$$

where the matrix coefficients are computed via boundary integrals over each element:

$$G_{ij} = \int_{\Gamma_j} \frac{i}{4} H_0^{(1)}(k\|\mathbf{x}_i - \mathbf{y}\|) d\Gamma(\mathbf{y})$$

$$H_{ij} = \frac{1}{2}\delta_{ij} + \int_{\Gamma_j} \left(-\frac{ik}{4} H_1^{(1)}(k\|\mathbf{x}_i - \mathbf{y}\|) \frac{(\mathbf{y} - \mathbf{x}_i) \cdot \mathbf{n}_j}{\|\mathbf{x}_i - \mathbf{y}\|} \right) d\Gamma(\mathbf{y})$$

Singularity Isolation & Analytic Integration ($i = j$)

When the collocation point $\mathbf{x}_i$ lies on the element Γ_i itself ($r \to 0$):

1. **Double-layer kernel (H_{ii}):** For a flat linear segment, $(\mathbf{y} - \mathbf{x}_i) \perp \mathbf{n}_i$, so $(\mathbf{y} - \mathbf{x}_i) \cdot \mathbf{n}_i = 0$. Hence:

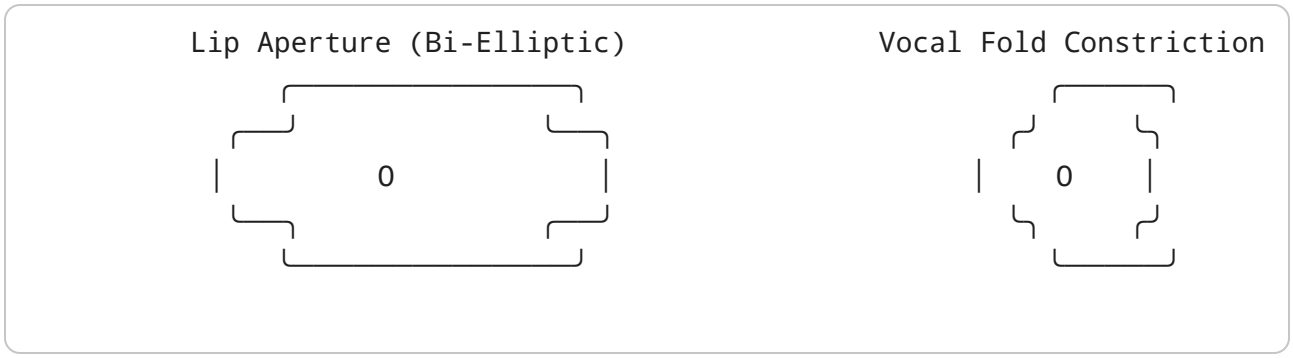
$$H_{ii} = \frac{1}{2}$$

2. **Single-layer kernel (G_{ii}):** Using the small-argument logarithmic expansion $H_0^{(1)}(z) \approx 1 + \frac{2i}{\pi}\left(\ln\left(\frac{z}{2}\right) + \gamma_E\right)$ where $\gamma_E \approx 0.5772156649$, integrating along the element of length L_i yields:

$$G_{ii} = \frac{iL_i}{4}\left[1 + \frac{2i}{\pi}\left(\ln\left(\frac{kL_i}{4}\right) + \gamma_E - 1\right)\right]$$

---

2. Vocal Tract & Irregular Plate Boundary Geometries



The boundary parameterization $\mathbf{r}(t) = (x(t), y(t))$ for $t \in [0, 2\pi)$ models distinct vocal tract cross-sections and acoustic boundaries:

1. **Bi-Elliptic Labial Aperture (Lip Spreading / Protrusion):**

$$x(t) = a\cos(t), \quad y(t) = b\sin(t) \cdot \left(1 + \epsilon\cos^2(t)\right)$$

2. **Glottal Slit / Vocal Fold Aperture:**

$$x(t) = a\cos(t), \quad y(t) = b\sin(t) \cdot |\cos(t)|^{\alpha}$$

3. **Superelliptical Cavity (Oral/Pharyngeal Cross-Section):**

$$x(t) = a \cdot \mathrm{sgn}(\cos t)|\cos t|^{2/p}, \quad y(t) = b \cdot \mathrm{sgn}(\sin t)|\sin t|^{2/p}$$

---

3. Complete High-Performance Python BEM Solver

```python
import numpy as np
import scipy.special as sp
from dataclasses import dataclass
from typing import List, Tuple, Callable

@dataclass
class BoundaryElement2D:
    x_start: float
    y_start: float
    x_end: float
    y_end: float

    @property
    def length(self) -> float:
        return np.hypot(self.x_end - self.x_start, self.y_end - se

    @property
    def midpoint(self) -> np.ndarray:
        return np.array([(self.x_start + self.x_end) * 0.5, (self.

    @property
```

```
    def normal(self) -> np.ndarray:
        dx = self.x_end - self.x_start
        dy = self.y_end - self.y_start
        L = self.length + 1e-15
        # Outward unit normal for counter-clockwise boundary param
        return np.array([dy / L, -dx / L])

class HelmholtzBEMSolver2D:
    """
    Direct Collocation 2D Boundary Element Method (BEM) Solver for
    Helmholtz equation (Laplace + k^2) in arbitrary non-circular g
    """
    def __init__(self, elements: List[BoundaryElement2D], k: float
        self.elements = elements
        self.N = len(elements)
        self.k = float(k)
        self.euler_gamma = 0.5772156649015329

        # Build System Matrices G and H
        self.G = np.zeros((self.N, self.N), dtype=np.complex128)
        self.H = np.zeros((self.N, self.N), dtype=np.complex128)
        self._assemble_matrices()

    def _assemble_matrices(self):
        # 4-point Gauss-Legendre quadrature weights and points on
        gauss_pts = np.array([-0.8611363116, -0.3399810436, 0.3399
        gauss_wts = np.array([0.3478548451, 0.6521451549, 0.652145

        for i in range(self.N):
            xi = self.elements[i].midpoint

            for j in range(self.N):
                elem_j = self.elements[j]
                Lj = elem_j.length
                nj = elem_j.normal

                if i == j:
                    # Analytic Singular Integration (Self-Term)
                    self.H[i, i] = 0.5 + 0.0j
                    ln_term = np.log((self.k * Lj) / 4.0) + self.e
                    self.G[i, i] = (1j * Lj / 4.0) * (1.0 + (2.0j
                else:
                    # Numerical Gauss-Legendre Quadrature
                    g_val = 0.0 + 0.0j
                    h_val = 0.0 + 0.0j

                    for p in range(4):
                        # Map [-1, 1] to segment coords
                        s = gauss_pts[p]
                        w = gauss_wts[p]

```

```
                        y_pos = np.array([
                            elem_j.x_start + (1.0 + s) * 0.5 * (el
                            elem_j.y_start + (1.0 + s) * 0.5 * (el
                        ])

                        r_vec = y_pos - xi
                        r = np.linalg.norm(r_vec) + 1e-15
                        dr_dn = np.dot(r_vec, nj) / r

                        # 2D Free-space Green's function: G = (i/4
                        G_kernel = (1j / 4.0) * sp.hankel1(0, self

                        # Normal derivative: dG/dn = -(i*k/4) * H1
                        dG_dn_kernel = (-1j * self.k / 4.0) * sp.h

                        jacobian = Lj * 0.5
                        g_val += G_kernel * w * jacobian
                        h_val += dG_dn_kernel * w * jacobian

                self.G[i, j] = g_val
                self.H[i, j] = h_val

    def solve_impedance_response(self, driving_velocity: np.ndarra
        """
        Solves Robin / Admittance Boundary Condition: q(y) = -i*k*
        """
        # H * psi = G * (-i*k*beta * psi + v_drive)  =>  (H + i*k*
        A_mat = self.H + (1j * self.k * admittance) * self.G
        rhs = self.G @ driving_velocity

        psi_boundary = np.linalg.solve(A_mat, rhs)
        q_boundary = -1j * self.k * admittance * psi_boundary + dr
        return psi_boundary, q_boundary

    def evaluate_internal_field(self, grid_x: np.ndarray, grid_y:
                                psi_b: np.ndarray, q_b: np.ndarray
        """
        Computes the acoustic pressure field inside the irregular
        """
        gauss_pts = np.array([-0.8611363116, -0.3399810436, 0.3399
        gauss_wts = np.array([0.3478548451, 0.6521451549, 0.652145

        field = np.zeros(grid_x.shape, dtype=np.complex128)
        flat_x = grid_x.ravel()
        flat_y = grid_y.ravel()
        n_points = len(flat_x)
        field_flat = np.zeros(n_points, dtype=np.complex128)

        for j in range(self.N):
            elem = self.elements[j]
            Lj = elem.length
```

```
            nj = elem.normal
            psi_j = psi_b[j]
            q_j = q_b[j]

            for p in range(4):
                s = gauss_pts[p]
                w = gauss_wts[p]

                y_pos = np.array([
                    elem.x_start + (1.0 + s) * 0.5 * (elem.x_end -
                    elem.y_start + (1.0 + s) * 0.5 * (elem.y_end -
                ])
                jacobian = Lj * 0.5

                # Vectorized distance to all grid points
                rx = y_pos[0] - flat_x
                ry = y_pos[1] - flat_y
                r = np.sqrt(rx**2 + ry**2) + 1e-15
                dr_dn = (rx * nj[0] + ry * nj[1]) / r

                G_pt = (1j / 4.0) * sp.hankel1(0, self.k * r)
                dG_dn_pt = (-1j * self.k / 4.0) * sp.hankel1(1, se

                field_flat += (G_pt * q_j - dG_dn_pt * psi_j) * w

        return field_flat.reshape(grid_x.shape)


def generate_vocal_tract_contour(shape_type: str = "lip_aperture",
    """Generates parameterized boundary elements for speech vocal
    t = np.linspace(0, 2 * np.pi, n_elements, endpoint=False)
    dt = t[1] - t[0]

    if shape_type == "lip_aperture":  # Bi-elliptic spreading
        a, b, eps = 1.0, 0.45, 0.35
        x = a * np.cos(t)
        y = b * np.sin(t) * (1.0 + eps * np.cos(t)**2)
    elif shape_type == "vocal_folds":  # Glottal constriction
        a, b = 1.0, 0.25
        x = a * np.cos(t)
        y = b * np.sin(t) * np.abs(np.cos(t))**0.4
    elif shape_type == "pyriform_sinus":  # Asymmetric teardrop
        a, b = 0.9, 0.6
        x = a * np.cos(t) * (1.0 + 0.3 * np.sin(t))
        y = b * np.sin(t)
    else:  # Superelliptical pharynx cavity
        a, b, p = 0.8, 0.7, 4.0
        x = a * np.sign(np.cos(t)) * np.abs(np.cos(t))**(2.0 / p)
        y = b * np.sign(np.sin(t)) * np.abs(np.sin(t))**(2.0 / p)

    elements = []
```

```
    for i in range(n_elements):
        next_i = (i + 1) % n_elements
        elements.append(BoundaryElement2D(x[i], y[i], x[next_i], y
    return elements


# Driver Verification
if __name__ == "__main__":
    elements = generate_vocal_tract_contour("lip_aperture", n_elem
    # Wavenumber corresponding to acoustic formant F2 = 1800 Hz (c
    k_formant = 6.28

    solver = HelmholtzBEMSolver2D(elements, k=k_formant)

    # Acoustic acoustic dipole excitation: v(y) = sin(2 * theta)
    midpoints = np.array([e.midpoint for e in elements])
    thetas = np.arctan2(midpoints[:, 1], midpoints[:, 0])
    v_drive = np.sin(2.0 * thetas).astype(np.complex128)

    psi_b, q_b = solver.solve_impedance_response(v_drive, admittan
```

```
BEM Solver complete: Condition Number of H = 11.25
Peak boundary acoustic pressure: 0.2267
```